Mittuniversitetet
MID SWEDEN UNIVERSITY

Försättsblad Prov Original

Kurskod	Provkod	Tentamensdatum
DT019G	T107	2025-06-05
Kursnamn	Datateknik GR (A), Objektbaserad programmering i C++	
Provnamn	Skriftligt teoriprov	
Ort	Sundsvall	
Termin		
Ämne		

6895642268

Utskriven 2025-11-27 kl. 14:12:10

Mittuniversitetet
MID SWEDEN UNIVERSITY

Tentamen

DT019G Objektbaserad programmering i C++
DT026G Objektbaserad programmering

Jimmy Åhlander

Instruktioner

Tid	3 timmar
Resurser	Laptop, mus, penna, sudd och tillhandahållet papper
Poäng	100
Preliminära betygsgränser	G 65 %
Allmänt	För uppgifter av typen flersvar ges poäng för varje korrekt markerat svar och minuspoäng för varje inkorrekt markerat svar. Minimum för en enskild uppgift är alltid noll poäng. Poängfördelningen är inte alltid jämn mellan alternativen i en uppgift. Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar. Om inget annat är angivet så ska uppgifterna alltid besvaras inom kontexten för C++. Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt om du inte redan lämnat in tentamen. InsperaScan används ej för denna tentamen.

Mittuniversitetet
Institutionen för data- och elektroteknik (DET)

Jimmy Åhlander, 010 - 142 86 90

1(a) Vilka av följande rader deklarerar giltiga konstruktorer för klassen Boo?

Välj ett eller flera alternativ:

- [ ] Boo m;
- [ ] Boo();
- [ ] Boo(Moo);
- [ ] Boo(Boo);
- [ ] Boo(Boo&);
- [ ] Boo(double);
- [ ] ~Boo();

Totalpoäng: 5

1(b) Vilka av följande påståenden är sanna efter att följande kod exekverat?

```
int p, *q = new int(3);
p = *q;
q = &p;
delete q;
q = nullptr;
```

Välj ett eller flera alternativ:

- [ ] Koden ger upphov till odefinierat beteende.
- [ ] p blir en dangling pointer.
- [ ] q blir en dangling pointer.
- [ ] p blir en wild pointer.
- [ ] q blir en wild pointer.
- [ ] En minnesläcka uppstår.
- [ ] Bara ett av de tre allokerade talen i `q` blir frigjort.

Totalpoäng: 5

1(c) Vad blir utskriften från en körning av följande kod?

```
std::string s = "Dalgatan 1, 852 33; SUNDSVALL";
std::cout << s.substr(s.find(",") + 2, s.find(";"));
```

Välj ett alternativ:

- `52 33`
- `, 852 33;`
- `852 33;`
- `852 33`
- `, 852 33`
- `852 33; SUNDSVALL`
- `Dalgatan 1, 852 33; SUNDSVALL`

Totalpoäng: 5

1(d) Vad sker vid följande kodrad?

```
Maze m();
```

Välj ett alternativ:

- Ett objekt m av Maze skapas med defaultkonstruktorn.
- Ett objekt m av Maze skapas med den syntetiserade defaultkonstruktorn.
- En funktion m som returnerar en Maze deklareras.
- En funktion m anropas utan argument.
- En klass Maze m deklareras.

Totalpoäng: 5

1(e) Vilken nivå av undantagssäkerhet garanterar att inga sidoeffekter uppstår om ett undantag kastas?

Välj ett alternativ:

- No-throw guarantee
- No exception guarantee
- Basic exception guarantee
- Medium exception guarantee
- Strong exception guarantee

Totalpoäng: 5

1(f) Vad är `std::string::npos`? Se nedanstående kod för inspiration.

```
if (person.name.find(name_part) != std::string::npos) {
    persons++;
}
```

Välj ett alternativ:

- npos är en `string` som är tom.
- npos är en `unsigned int` med sista positionen i strängen.
- npos är en `char` med sista bokstaven i strängen.
- npos är en `int` med antalet matchningar.
- npos är en `size_t` med största möjliga värde.

Totalpoäng: 5

1(g) Vilken av följande konventioner är bäst att följa för inkapsling av klassmedlemmar?

Välj ett alternativ:

- Alla medlemmar ska vara publika.
- Alla medlemsvariabler ska vara privata och alla medlemsfunktioner ska vara publika.
- Alla medlemmar ska vara privata, om det inte finns anledning till annat.
- Alla medlemmar ska vara privata.
- Medlemmar behöver inte inkapslas alls, det sköter klasser och structar (på olika sätt) som standard.

Totalpoäng: 5

1(h) Anta att du har följande vektor v:

```
std::vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Anta nu att du lägger till ett element till v. Vilken *kapacitet* (*capacity*) kommer vektorn ha efter tillägget?*

Välj ett alternativ:

- 0
- 1
- 9
- 10
- 11
- 15

** Svaret är implementationsspecifikt och utgår ifrån MSVC men kan även härledas från GCCs och CLANGs implementation.*

Totalpoäng: 5

1(i) Varför fungerar inte följande uttryck om `b` är en pekare till ett objekt med medlemsvariabeln `a`?

```
*b.a
```

Välj ett alternativ:

- För att medlemsoperatorn `.` har högre prioritet än operatorn `*`.
- För att ordningen är omkastad och ska istället vara `*a.b`
- För att `*` syftar till pekare och kan därmed endast användas för pekare.
- För att objekt som pekas på inte kan ha medlemsvariabler.
- För att `a` skulle kunna vara en konstant.

Totalpoäng: 5

1(j) Vilka av följande konstruktorer skulle kunna anropas för kodraden nedan?

```
Bar b;
```

Välj ett eller flera alternativ:

- `Bar();`
- `Bar(Bar* b = nullptr);`
- `Bar(const Bar& other);`
- `Bar(X, Y);`
- `Bar(int);`
- `explicit Bar(int x = 5);`

Totalpoäng: 5

1(k) Vad blir konsekvensen av att en överlagrad utströmsoperator inte returnerar något? Se nedan.

```
void operator<<(...)
    // code
}
```

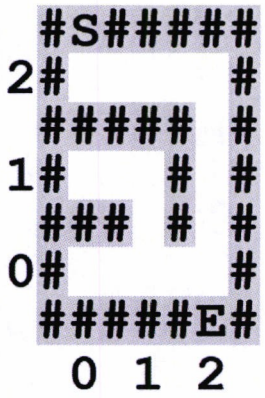
Välj ett alternativ:

- Det kommer inte vara möjligt att kedja användandet av operatorn för den givna klassen eller structen.
- Det är inte möjligt att överlagra utströmsoperatorn utan att returnera *något*.
- Utströmmen (som läses in som argument) kommer att stängas.
- Det är inte möjligt att överlagra utströmsoperatorn utan att returnera strömmen.
- Inget kommer att skrivas på utströmmen då den aldrig returneras.

Totalpoäng: 5

1(I) Följande bild visar en 3x3-labyrint bestående av 9 noder genererad med *depth-first search* (DFS). Vid vilken koordinat startade DFS genereringen av labyrinten?

Koordinaterna är uttrycka på formen (x, y), motsvarande (kolumn, rad).



Välj ett alternativ:

- (0,0)
- (0,1)
- (0,2)
- (1,0)
- (1,1)
- (1,2)
- (2,0)
- (2,1)
- (2,2)

Totalpoäng: 5

i

Inför uppgifter där kod tolkas

I de kommande uppgifterna tolkar och evaluerar du kod som någon annan producerat. Läs uppgifterna noga då målet kan skilja sig mellan dem. Det kan handla om att identifiera brister i koden eller att besvara en särskild fråga.

I vissa fall anropas funktioner vars kod inte presenteras. I de fallen kan du anta att de anropade funktionerna fungerar ändamålsenligt och returnerar den mest rimliga datatypen givet kontexten.

2(a) Klassen **`linked_list`** representerar en länkad lista vars noder håller flyttal. Objekt av klassen håller två pekare: head och tail som leder till första respektive sista noden i listan. Varje nod håller två pekare: next och prev som leder till nästa nod (närmare svansen) respektive föregående nod (närmare huvudet) samt ett flyttal **`f`**. Nedan följer koden för medlemsfunktionen **`push_back`** som lägger till en ny nod längst bak i listan.

```
void linked_list::push_back(float f) {
    node* n = new node();

    if (is_empty()) {
        head = tail = n;
        head->f = f;
        return;
    }

    tail->next = n;
    n->prev = tail;
    n = tail;
}
```

Vilka problem har funktionen?

Välj ett eller flera alternativ:

- ☐ Specialfallet där listan är tom hanteras inkorrekt eller inte alls.
- ☐ Specialfallet där listan redan håller exakt en nod hanteras inkorrekt eller inte alls.
- ☐ Fallen där listan redan håller flera noder hanteras inkorrekt eller inte alls.
- ☐ Nodens flyttal tilldelas aldrig i vissa situationer.
- ☐ I vissa fall kan minnesläckor uppstå, t.ex. om push_back anropas flera gånger på samma lista.

Totalpoäng: 5

2(b) Vad är problemet med nedanstående operator?

```
linked_list& linked_list::operator+=(const linked_list& list) {
    for (node* p = list.head; p != nullptr; p = p->next) {
        this->push_back(p->f);
    }
    return *this;
}
```

Välj ett alternativ:

- Det går inte att blanda + och = som operator.
- push_back borde anropas på list, inte på this.
- En krasch uppstår om en icke-konstant lista används som argument.
- En oändlig loop kan uppstå om operatorn anropas på samma lista som används som argument.
- Det sista elementet från list kopieras inte.
- Bara hälften av elementen från list kopieras.
- Returtypen är fel.

Totalpoäng: 5

Inför uppgifter där kod kompileras

I de kommande uppgifterna skriver du kod som kompileras, testkörs och valideras genom ett antal tester. För dessa uppgifter:

- Används C++14 (gcc-5 5.1.1). Observera att funktionalitet exklusiv till C++17, C++20 eller nyare därför inte kan användas. I normalfallet bör detta inte påverka dig nämnvärt.
- Skrivs all kod i motsvarigheten till en fil. Det finns ingen uppdelning i header- och cpp-filer. Du kan skriva dina deklarationer och definitioner i samma textfält eller helt enkelt skriva skriva dina definitioner inline.
- Måste samtliga testfall i varje uppgift valideras för att erhålla full poäng. Validerade tester innebär dock inte per automatik att du får poäng, exempelvis ger inte hårdkodad utdata poäng.
- Måste du endast göra utskrifter där du instruerats att göra så. Extra utskrifter som inte efterfrågats t.ex. "Hej och välkommen till mitt program! Skriv in ett tal" kommer invalidera de automatiska testerna.
- Tillåts inte globala variabler. Globala konstanter är acceptabla.

3 Implementera en klass MyString. Ett objekt i klassen MyString ska kunna hålla en dynamiskt allokerad C-sträng, d.v.s. `char*`.

MyString ska ha följande medlemmar:

- En medlemsvariabel `ptr` av typen `char*`.
- En medlemsvariabel `len` av typen `unsigned int`.
- En parametriserad konstruktor med två inparametrar: en av typen `const char*` och en av typen `unsigned int` för antalet tecken.
- En destruktor.
- En överlagrad tilldelningsoperator (=).
- En funktion `set` som sätter värdet på strängen `ptr` pekar på.
- En funktion `get` som returnerar värdet på strängen `ptr` pekar på.

Medlemmarna ska inkapslas enligt konvention. Minnesläckor måste undvikas. All kopiering som sker måste vara djup. Du behöver inte visa hänsyn till regeln the big three, d.v.s. du behöver inte implementera någon annan funktion än de listade ovan för full poäng.

För att testa din klass skapar du två objekt `mystr1` och `mystr2` samt anropar funktioner enligt kommentarerna nedan. För din bekvämlighet är dessa tester nästintill färdigställda redan. Det enda du behöver göra är att justera de tre utkommenterade raderna kod som innehåller tre punkter (...).

Observera att du *inte* får använda `std::basic_string` eller någon av dess specialiseringar så som `std::string`.

Testfall #	Input	Förväntad output
1	5 Hello 6 World! 3 Bye	Hello World! World! Bye
2	4 Test 4 Text 5 Works	Test Text Text Works

```
#include <iostream>
int main() {
    char *char_arr1, *char_arr2, *char_arr3;
    unsigned int len1, len2, len3;

    // läs in antalet tecken för första strängen, och
    // allokera därefter en ny dynamisk array med den inlästa storleken + 1 (se tips nedan)
        och läs in från stdin.
    std::cin >> len1;
    // char_arr1 = ...
    std::cin >> char_arr1;

    MyString mystr1(char_arr1, len1);


    // upprepa samma inläsningsprocess för den andra strängen
    std::cin >> len2;
    // char_arr2 = ...
    std::cin >> char_arr2;

    MyString mystr2(char_arr2, len2);

    // skriv ut båda strängarna med anrop till get
    std::cout << mystr1.get() << '\n';
    std::cout << mystr2.get() << '\n';

    mystr1 = mystr2;

    // upprepa samma inläsningsprocess för den tredje strängen
    std::cin >> len3;
    // char_arr3 = ...
```

Testkod

Tips:

- Objektets pekare ptr ska peka på en dynamisk array av tecken.
- Kom ihåg att storleken på arrayen du allokerar alltid blir antalet synliga tecken i ordet plus ett, på grund av nulltecknet ('\0') som avslutar varje sträng. För ett ord med fem synliga tecken, t.ex. "Hello", behöver du alltså

allokera plats för sex tecken ('H', 'e', 'l', 'l', 'o', '\0').

- Kom ihåg att kopiera över samtliga tecken inklusive nulltecknet i de funktioner som kopierar.

Totalpoäng: 15

4 Implementera en rekursiv funktion som räknar antalet binära strängar av längd n utan konsekutiva ettor.

T.ex. för följande längder får du följande binära strängar:
n = 1 → 0 1 → 2 stycken.
n = 2 → 00 01 10 → 3 stycken.
n = 3 → 000 001 010 100 101 → 5 stycken.

Observera att binära strängar med konsekutiva ettor, d.v.s. där två ettor kommer i följd, *inte* räknas med, t.ex. 11 för n = 2, eller 011, 110, 111 för n = 3.

Testfall #	Input	Förväntad output
1	2	3
2	3	5
3	4	8

```
// kom ihåg dina includes
unsigned int count_binary_strings(unsigned int n, std::string s = "") {
    // din lösning här
}

int main() {

    // läs in talet från stdin och ersätt det hårdkodade argumentet
    std::cout << count_binary_strings(3);

    return 0;
}
```

Testkod

Observera:

- Lösningen måste vara rekursiv där rekursionen är av väsentlig betydelse för lösningen.
- Loopar får inte användas för att kringgå rekursionen.
- Globala variabler får inte förekomma.

Tips:

- Funktionen implementeras enklast med två rekursiva anrop.
- Du kan börja med en tom sträng som du konkatenerar en nolla eller etta till, steg för steg, i respektive anrop. Var särskilt försiktig med när du lägger till en etta så att du inte skapar en otillåten binär sträng.
- Funktionens signatur ovan är bara ett förslag och får ändras.
- Du kan använda strängfunktionen `at` eller indexoperatorn `[]` för hämta ett tecken i en sträng. Kom ihåg att tecken (char) skrivs med enkla citattecken '1' och strängar med dubbla citattecken "1".

- För den med ett skarpsynt matematiskt öga finns det en helt annorlunda, fortfarande rekursiv, approach till uppgiften.

Totalpoäng: 15